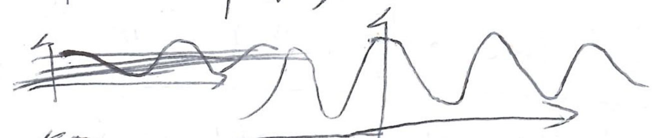
Learning Rate decay types

exponential decay: old CNN / DNN

②

cosine decay: small LM

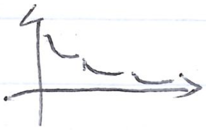


③

WSD ~~cosine~~ decay: medium LM

④

chanchiall WSD decay: large LLM



⑤

linear decay: ~~chh~~ chinchilla

①

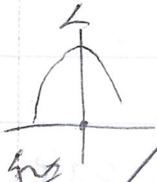
---

scaling predicts final model loss

↓ the fitted scaling models (generally) accurately predict the final model losses

① : LLama3 (2024) scaling laws: isoflops-style scaling (≈ 29-1 ratio)
(ratio = token-param)

② : Hunyuan-1 (2024) large scaling laws: yet more isoflops-styles scaling (but for MOE params size)



↓ isoflops scaling law: adjust params + data size, but total flops constant (↓ cost / ~~Given~~ computation)

- chinchilla scaling law: optimal token-param ratio
- ~~Tran~~ Kalapan scaling laws: compute-model optimized

↓ Given a fixed compute budget, what's the optimal trade-off between model size and datasize?

2020 ⇓ Kaplan's scaling law (2020): bigger is better: increase compute to increase accuracy / reduce loss (⇑ exponential LR)

2022 ⇓ chinchilla scaling law (2022) (deepmind): smarter is better: fixed compute = balance between params + data. (⇑ cosine LR decay; compute-optimal scaling)

2022 ↓ 2024 ⇓ isoFlops scaling law: efficiency is everything, optimal allocation along compute lines (⇑ multi-step WSD)

8. Minimax-01 (2025):

Architecture scaling law + Chinchilla method

Current summary: for case study.

CerebrasGPT:

- use muP to make hyperparams invariant to scale
- Directly use the chinchilla scaling formula.

DeepSeek recipe:

- Assume most transformer hypers are invariant to scale.
- Do a scaling analysis on batch / LR to figure out optimal scaling sizing
- IsoFLOP analysis to figure out model
  → use a piecewise-linear schedule to make chinchilla scaling cheap.

MiniCPM recipe:

- use muP to make transformer + LR invariant to scale
- use a piecewise linear schedule to get sample for chinchilla method (curve fitting)

LLaMA 3 / Hunyuan

- Just isoflops (no other scaling details)

Minimax: Architecture choice / decision scaling